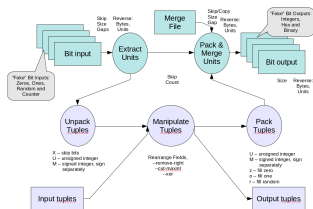


bdd — Bit Data Dump & Stream Engine

The Swiss Army knife for bitstreams. Slice, transform, arithmetic-scale, bitwise-mask, fuse, project, and transcode unaligned binary streams of arbitrary bit widths at multi-gigabit speeds.

Originally created by Esa Turtiainen in Python 2 (2010), 2026 `bdd` was re-engineered in modern safe Rust for memory safety, zero external C dependencies, and line-rate multi-gigabit throughput.

Available as **Fedora / RHEL RPM**, **Debian / Ubuntu DEB**, **Rust library crate**, **C SDK**, **Python SDK**, and native **MCP Server**.



1. Why bdd? The Byte-Boundary Trap vs. Single-Bit Reality

The Problem: Standard Tools are Trapped at Byte Boundaries

Most Unix tools (`dd` , `hexdump` , `od` , standard shell pipes) operate strictly on byte (8-bit) or machine-word (16, 32, 64-bit) boundaries. When you run `dd bs=1` , you are still constrained to 8-bit multiples.

The Reality: Real-World Data Lives at Arbitrary Bit Widths

Real-world binary data does not align neatly to 8-bit bytes:

- **IoT & Embedded Telemetry:** 12-bit ADC raw samples, 10-bit DAC calibration values, 1-bit status flags.
- **Broadcast Multimedia:** 13-bit PIDs in MPEG-TS 188-byte containers, unaligned 32-bit MP3 frame headers, 4-bit JPEG chroma nibbles.
- **Modern AI Inference:** NVIDIA Blackwell 4-bit floats (NVFP4 E2M1), OCP Microscaling 6-bit floats (FP6 E3M2), OCP 8-bit floats (FP8 E4M3/E5M2), Brain Float 16 (BF16).
- **Network Headers:** IPv4 4-bit IHL, TCP 1-bit control flags (`SYN` , `ACK` , `FIN`), unaligned DSCP/ECN QoS fields.
- **Kernel Internals:** Linux Netlink connector sockets, `/proc/[pid]/pagemap` 64-bit page table entries with 55-bit PFNs and discrete single-bit flags.

Attempting to slice, manipulate, or transcode these unaligned fields with standard shell scripts or ad-hoc C programs requires dozens of lines of brittle manual bitshifts, bitwise masks, and fragile endianness adjustments.

The Solution: Single-Bit Stream Transformation

`bdd` brings sub-byte surgical precision to the Unix pipeline. What previously required complex C structs or slow Python glue scripts now happens in a single, intuitive stream arrow expression:

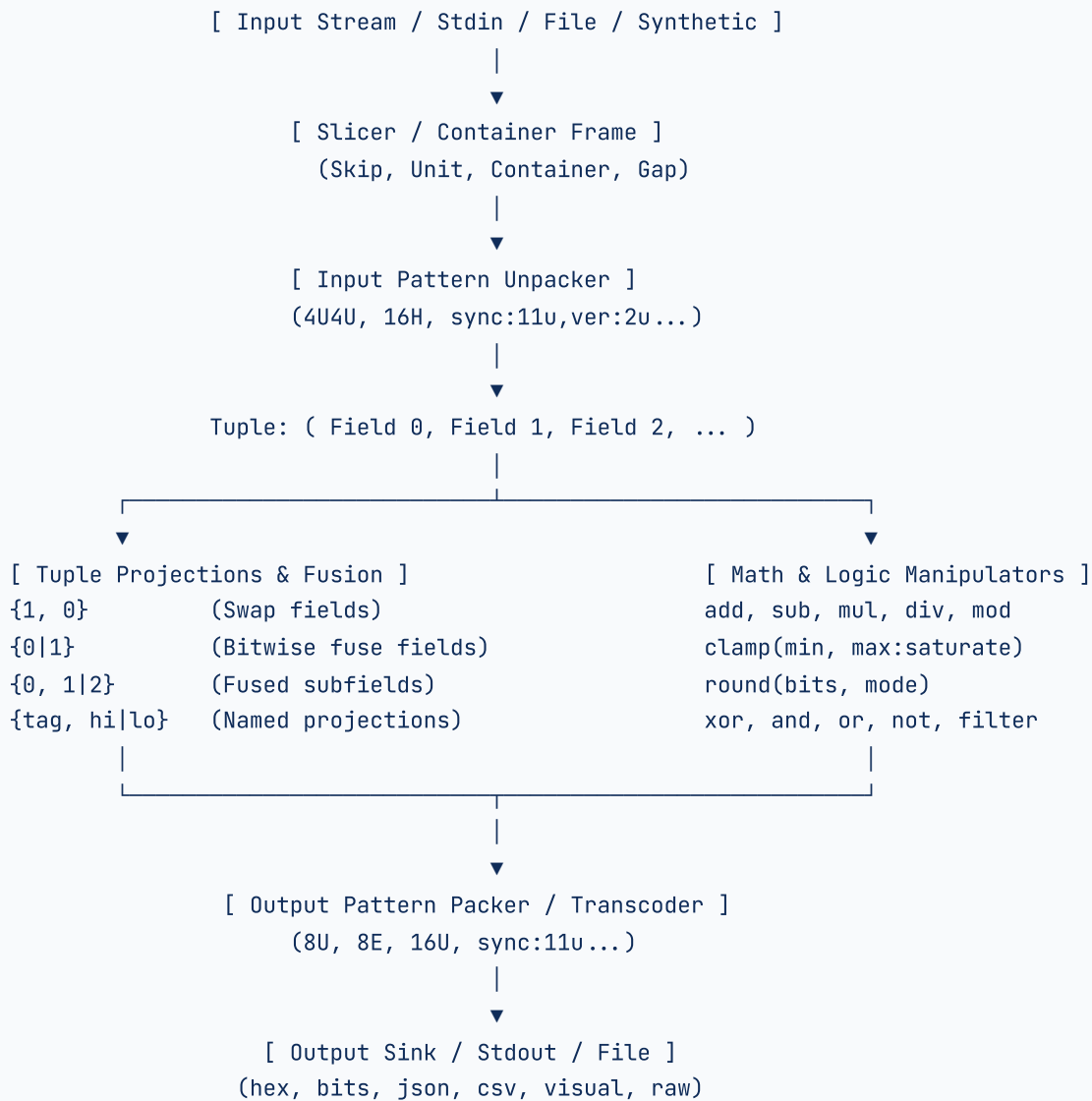
```
# Slices two 4-bit nibbles, fuses them into an 8-bit byte, and formats as hex:
echo -ne "\xfa" | bdd "4U4U → {0|1} → 8U → hex"
# Output: fa

# Swap high and low 4-bit nibbles on the fly:
echo -ne "\xfa" | bdd "4U4U → {1, 0} → 4U4U → hex"
# Output: af

# Quantize FP16 neural network weights directly to NVIDIA Hopper OCP FP8 (E4M3):
bdd "16H → 8E" < model_fp16.bin > model_fp8.bin

# Calibrate 12-bit ADC sensor data, remove DC bias, amplify, and clamp to 8-bit DAC:
bdd "12 → sub(512) → mul(2) → clamp(0, 255:saturate) → 8" < adc.raw > dac.raw

# Extract 13-bit MPEG-TS PIDs from repeating 188-byte container frames:
bdd "188B[11:13] → 13" < stream.ts
```



2. The Core Paradigm: Unified Stream Arrow Pipelines

The fundamental interface of `bdd` is the **Stream Arrow Transformation Pipeline**:

```
[source → ] [container / slicer → ] [pattern → ] [manipulator...] [ → pattern] [ → slicer] [ → sink]
```

Instead of passing dozens of disconnected CLI flags, you describe the entire lifecycle of your bitstream in a single readable pipeline string:

```
bdd "stdin → 8 → 4U4U → {0|1} → 16U → 16 → stdout"
```

Zero Redundant Dimensions: Automatic Bit-Width Derivation

In `bdd`, **pattern declarations automatically define input and output unit bit sizes**.

You do **not** need to specify redundant `--input-unit` or `--output-unit` flags:

- Declaring `16H` automatically sets the input unit size to **16 bits**.
- Declaring `4U4U` automatically sets the unit size to **8 bits** (4 + 4).
- Declaring `8E` automatically sets the output unit size to **8 bits**.
- Declaring `sync:11u,ver:2u,layer:2u` automatically sets the unit size to **15 bits**.

Therefore, the explicit pipeline above simplifies cleanly to:

```
bdd "stdin → 4U4U → {0|1} → 16U → hex"
```

Or when using default standard input and output:

```
bdd "4U4U → {0|1} → 16U → hex"
```

Built-in Stream Sources

Sources specify where raw bits originate:

SOURCE	DESCRIPTION	EXAMPLE
<code>stdin</code>	Standard input pipe or redirected file (default)	<code>bdd "stdin → 8 → hex"</code>
<code>zeros</code>	Infinite stream of continuous zero bits (<code>00000000...</code>)	<code>bdd "zeros → 8 → hex" --count 4</code>
<code>ones</code>	Infinite stream of continuous one bits (<code>11111111...</code>)	<code>bdd "ones → 8 → hex" --count 4</code>
<code>rand</code>	High-entropy random bits from <code>/dev/urandom</code>	<code>bdd "rand → 16 → hex" --count 2</code>
<code>counter</code>	Monotonically increasing sequential numbers (<code>0,1,2,3...</code>)	<code>bdd "counter → 8 → hex" --count 4</code>
<code>netlink</code>	Live Linux kernel process lifecycle events (<code>AF_NETLINK</code> <code>CN_IDX_PROC</code>)	<code>sudo bdd "netlink → json"</code>
<code>file(path)</code>	Direct file read without shell redirection	<code>bdd "file('dump.bin') → 8 → hex"</code>
<code>tuples</code>	Reads comma-separated text tuple lines from standard input	<code>bdd "tuples → 8U8U → stdout"</code>

Built-in Stream Sinks

Sinks specify how the processed bits or tuples are formatted on output:

SINK	DESCRIPTION	EXAMPLE OUTPUT
<code>stdout</code>	Raw binary bitstream written to standard output (default)	Binary bytes
<code>hex</code>	Space-delimited lowercase hexadecimal byte/word strings	<code>00 fa</code>
<code>bits</code>	Clean ASCII bit strings (<code>0</code> and <code>1</code>)	<code>11111010</code>
<code>json</code>	Newline-delimited JSON arrays (NDJSON)	<code>[15, 10]</code>
<code>json:object</code>	Keyed JSON dictionaries using pattern field names	<code>{"sync":71,"pid":256}</code>
<code>csv</code>	Standard comma-separated values	<code>15,10</code>
<code>visual</code>	Interactive ANSI colorized terminal dump with bitfield borders	Color matrix
<code>integers</code>	Unsigned base-10 integer per unit (one per line)	<code>250</code>
<code>file(path)</code>	Direct file write without shell redirection	<code>bdd "... → file('out.bin')"</code>

Physical Containers & Framing Slicers: Solving the Offset Problem

Active fields rarely float in empty space; they reside inside physical containers (bytes, 16-bit halfwords, 32-bit registers, 188-byte MPEG packets).

Rather than forcing you to mentally calculate initial skips and trailing gaps, `bdd` provides declarative container notation:

1. Form A Containers: `raw[offset:unit]`

Extracts an inner payload of `unit` bits starting at `offset` inside a repeating outer container of `raw` bits:

- `8[2:4]` : Inside each 8-bit byte, extract 4 bits starting at bit offset 2 (skipping 2 bits before, and 2 bits after).
- `188B[11:13]` : Inside each 188-byte MPEG-TS packet, extract the 13-bit PID starting at bit offset 11 (automatically skipping the remaining 1,480 bits of payload).

- `32[0:16]` : Extract the 16-bit Left channel from a 32-bit stereo audio word.
- `32[16:16]` : Extract the 16-bit Right channel from a 32-bit stereo audio word.

2. Form B Containers: `[pre:unit:post]`

Expresses symmetrical framing by specifying pre-gap and post-gap explicitly:

- `[2:4:2]` : Skip 2 bits, extract 4 bits, skip 2 bits (repeating container of 8 bits).

3. Periodic Gaps & Initial Stream Skips

- **Initial Skip (`skip : unit`)**: Skip bits before the first unit is read. `123 : 8` skips 123 bits into the stream and begins streaming 8-bit units.
- **Periodic Gap (`unit + gap`)**: Skip bits after every extracted unit. `8 + 24` reads 8 bits, skips 24 bits, and repeats (extracting 1 byte out of every 4-byte word).
- **Full Unified Syntax**: `123 : 1888[11:13] + 8 → 13` (Initial skip of 123 bits, 188-byte container, 13-bit payload at offset 11, periodic inter-packet gap of 8 bits, emitted as 13-bit units).

3. Pattern Grammar & Tuples: Slicing Structured Fields

When data contains multiple heterogeneous fields, a single unit size is not enough. You unpack the unit into an ordered **Tuple of Fields** using pattern grammar:

```
bdd "sync:11u,version:2u,layer:2u,protect:1b → json:object" < audio.mp3
```

Supported Pattern Specifiers

CODE	TYPE	DESCRIPTION
U	Unsigned Integer (Big-Endian)	Most Significant Bit first across the field width.
u	Unsigned Integer (Little-Endian)	Bit-reversed within the field (LSB first).
S	Signed Integer (Two's Complement, Big-Endian)	Preserves sign bit at MSB.
s	Signed Integer (Two's Complement, Little-Endian)	Bit-reversed within the field.
M	Sign-Magnitude Integer (Big-Endian)	High bit is sign (0 = +, 1 = -), remaining bits are magnitude.
m	Sign-Magnitude Integer (Little-Endian)	Bit-reversed sign-magnitude integer.
b	Boolean Flag	1-bit boolean flag (evaluates to integer 0 or 1).
c / c	Raw Byte Character / Bytes	Preserves raw ASCII or byte characters without lossy UTF-8 conversion.
x	Discard / Padding Bits	Consumed from input stream and discarded (never emitted into tuple).
k	Constant Field	Verifies constant magic numbers or synchronization headers.

Floating-Point Codecs & The Universal f64 Currency

bdd includes zero-dependency pure-Rust codecs for modern AI, GPU, and DSP floats:

FLOAT CODEC	TOTAL BITS	EXPONENT / MANTISSA	STANDARD & USAGE
16F	16	5 exp / 10 man	IEEE 754 Half-Precision Float (binary16).
16H	16	5 exp / 10 man	IEEE 754 Half-Precision Float (alias for 16F).
BF16	16	8 exp / 7 man	Google Brain Floating Point (matches FP32 dynamic range).
8E	8	4 exp / 3 man	OCP FP8 (E4M3) — NVIDIA Hopper & Blackwell AI training.
8M	8	5 exp / 2 man	OCP FP8 (E5M2) — NVIDIA Hopper & Blackwell AI inference.
6E	6	3 exp / 2 man	OCP Microscaling FP6 (E3M2) — Sub-byte quantized LLM weights.
4E	4	2 exp / 1 man	NVIDIA Blackwell NVFP4 (E2M1) — 4-bit AI tensor core weights.

All float decoders transcode through standard 64-bit IEEE 754 floats (**f64**) as a universal currency. You can quantize or transcode between any two floating-point formats in a single pass:

```
# Quantize FP16 weights directly to NVIDIA Hopper FP8:
bdd "16H → 8E" < model_fp16.bin > model_fp8.bin

# Quantize FP16 weights to NVIDIA Blackwell 4-bit NVFP4:
bdd "16H → 4E" < model_fp16.bin > model_nvfp4.bin
```

4. In-Flight Transformations & Field Fusion

Once fields are unpacked into a tuple, you can project, rearrange, bitwise-fuse, arithmetic-scale, and filter them before packing them to output.

1. Shorthand Tuple Projection: **{indices...}**

Use curly braces **{...}** to select, reorder, or duplicate fields:

- {1, 0}** : Swaps field 0 and field 1 (e.g. **4U4U** → **{1, 0}** → **4U4U**).
- {0, 2}** : Emits field 0 and field 2, discarding field 1.
- {0, 0, 0}** : Duplicates field 0 three times.
- {tag, length}** : Projects named fields directly.

2. Bitwise Field Fusion: `{0|1}` and `{0, 1|2}`

Often you need to glue multiple unaligned fields back into a single wider integer or bitfield. Use the pipe operator `|`:

- `{0|1}` : Glues field 0 and field 1 together into a single packed integer. If field 0 is `4U ( 1111 = 0xF )` and field 1 is `4U ( 1010 = 0xA )`, `{0|1}` computes `(0xF << 4) | 0xA = 0xFA`.
- `{0, 1|2}` : Keeps field 0 unchanged, and fuses field 1 and field 2 into a single field.
- `{tag, hi|lo}` : Keeps `tag` separate, and fuses `hi` and `lo` nibbles together.
- **Explicit Bit-Widths (`{0:4|1:4}`)**: Specify bit-lengths explicitly when operating on raw untyped tuples.

3. Ordered Math & Logic Manipulators

Chain manipulators directly inside arrow pipelines:

MANIPULATOR	SYNTAX	DESCRIPTION
Add	<code>add(val)</code> or <code>add(field, val)</code>	Adds <code>val</code> to field (default: field 0).
Subtract	<code>sub(val)</code> or <code>sub(field, val)</code>	Subtracts <code>val</code> from field.
Multiply	<code>mul(val)</code> or <code>mul(field, val)</code>	Multiplies field by <code>val</code> .
Divide	<code>div(val)</code> or <code>div(field, val)</code>	Performs integer division of field by <code>val</code> .
Modulo	<code>mod(val)</code> or <code>mod(field, val)</code>	Computes field modulo <code>val</code> .
Bitwise XOR	<code>xor(mask)</code> or <code>xor(field, mask)</code>	Bitwise XOR with hex (<code>0xAA</code>) or decimal mask.
Bitwise AND	<code>and(mask)</code> or <code>and(field, mask)</code>	Bitwise AND with mask.
Bitwise OR	<code>or(mask)</code> or <code>or(field, mask)</code>	Bitwise OR with mask.
Bitwise NOT	<code>not</code> or <code>not(field)</code>	Inverts all bits of the field.
Shift Left	<code>shift_left(bits)</code>	Left-shifts field by N bits.
Shift Right	<code>shift_right(bits)</code>	Right-shifts field by N bits (discards low N bits).
Clamp	<code>clamp(min, max:mode)</code>	Clamps value to <code>[min, max]</code> (<code>saturate</code> , <code>wrap</code> , <code>zero</code> , <code>drop</code>).
Round	<code>round(bits, mode)</code>	Rounds floats or integers (<code>trunc</code> , <code>floor</code> , <code>ceil</code> , <code>round</code>).
Filter	<code>filter(field, op, val)</code>	Drops entire record if predicate is false (<code>=</code> , <code>≠</code> , <code><</code> , <code>≤</code> , <code>></code> , <code>≥</code>).

5. Fascinating Real-World Scenarios

Here is `bdd` in action across six high-impact systems engineering and data pipeline workflows:

Scenario 1: IoT Sensor Calibration & Dynamic Range Compression

Embedded 12-bit ADCs (analog-to-digital converters) frequently stream raw readings offset by an analog DC bias. In this pipeline, we ingest 12-bit ADC samples, subtract the 512-count DC offset, amplify the sensor signal by $2\times$, clamp values to prevent DAC saturation (0..255), and pack the result into clean 8-bit DAC bytes:

```
bdd "12 → sub(512) → mul(2) → clamp(0, 255:saturate) → 8" < adc_raw.bin > dac_out.bin
```

Scenario 2: Real-Time Broadcast Video MPEG-TS Descrambling

In digital video broadcasting (DVB / ATSC), MPEG-2 Transport Streams transmit unaligned 188-byte packets. Packet headers contain sync bytes (`0x47`), a 13-bit PID, and scrambling control flags. In this pipeline, we isolate the 184-byte payload (`188B[32:1472]`), apply on-the-fly XOR descrambling against a rotating sync key (`0x5A`), and repack the descrambled payload back into standard 188-byte containers:

```
bdd "188B[32:1472] → xor(0x5A) → 188B[32:1472]" < scrambled.ts > clear.ts
```

Scenario 3: Audio DSP Headroom Limiting & Dynamic Soft-Clipping

Digital studio masters recorded in 16-bit signed PCM (`16S` , range $-32768..+32767$) often require brickwall headroom limiting before transmission to prevent loudspeaker voice-coil blowout. This pipeline ingests 16-bit signed audio, clamps peaks strictly to $[-16000, +16000]$ with saturation, applies a 6 dB attenuation (`div(2)`), and repacks the output as bit-perfect signed PCM:

```
bdd "16S → clamp(-16000, 16000:saturate) → div(2) → 16S" < master.pcm > limited.pcm
```

Scenario 4: Network Packet QoS / DSCP & CoS Priority Rewriter

Network routers mark Quality of Service (QoS) inside the 8-bit Type of Service (ToS) field of IPv4 headers (RFC 791). The Differentiated Services Code Point (DSCP) occupies the upper 6 bits, while Explicit Congestion Notification (ECN) occupies the lower 2 bits. In this pipeline, we inspect IPv4 headers, isolate the ToS byte (`20B[8:8]`), rewrite the DSCP priority to Expedited Forwarding (`0x28` / EF Class), and re-emit the packet:

```
bdd "20B[8:8] → 8U → {0} → or(0x28) → 8U → 20B[8:8]" < incoming_packets.bin > qos_packets.bin
```

Scenario 5: Hardware Watchdog Heartbeat & Rolling Counter

Embedded watchdogs expect sequential rolling counter ticks where specific bits are periodically inverted to prove the micro-controller's ALU is not stuck. This pipeline generates an infinite hardware counter, slices each byte into high and low 4-bit nibbles (`4U4U`), inverts the high nibble with bitwise NOT, fuses the nibbles back together (`{0|1}`), and emits formatted hexadecimal words to stdout:

```
bdd "counter → 8 → 4U4U → {not(0), 1} → {0|1} → hex" --count 4
# Output: f0 e1 d2 c3
```

Scenario 6: Direct AI Model Quantization (FP16 → Hopper FP8 / Blackwell NVFP4)

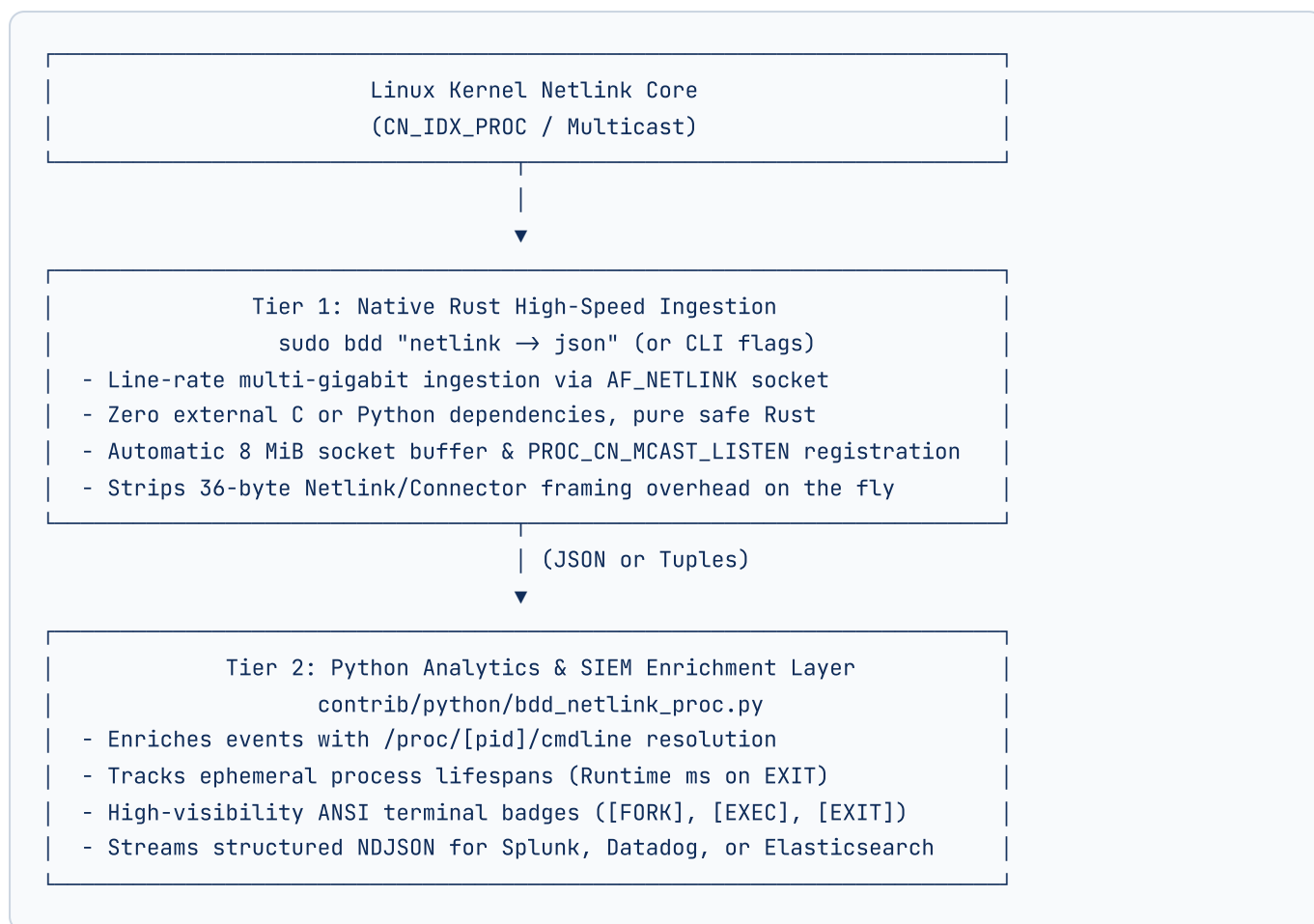
Quantizing large neural network checkpoints usually requires installing gigabytes of heavy Python frameworks (`torch` , `transformers` , `accelerate`). With `bdd` , you can quantize raw FP16 weights directly to NVIDIA Hopper OCP FP8 or NVIDIA Blackwell 4-bit NVFP4 at line-rate multi-gigabit speeds using zero Python dependencies:

```
# Quantize half-precision FP16 weights to NVIDIA Hopper FP8 (E4M3):
bdd "16H → 8E" < weights_fp16.bin > weights_fp8.bin

# Quantize half-precision FP16 weights to NVIDIA Blackwell 4-bit NVFP4 (E2M1):
bdd "16H → 4E" < weights_fp16.bin > weights_nvfp4.bin
```

6. Native Linux Netlink Kernel Telemetry vs. Python Analytics

Modern Linux kernels provide real-time process execution, fork, exit, and credential events via the Netlink Process Connector (`AF_NETLINK` , `CN_IDX_PROC`). `bdd` provides a two-tier architecture for kernel telemetry:



1. Tier 1: Native Rust Kernel Ingestion (`bdd "netlink → ..."`)

Run `bdd` with root privileges to connect directly to the kernel process connector:

```
# Ingest kernel process events directly into structured JSON:
sudo bdd "netlink → json"

# Or using dedicated CLI flags:
sudo bdd --input-netlink --output-json

# Unpack specific proc_event header fields (Timestamp, CPU ID, Event Code):
sudo bdd --input-netlink "timestamp_ns:64U,cpu:32U,what:32U" --output-tuples
```

2. Tier 2: Python Analytics & SIEM Enrichment (`bdd_netlink_proc.py`)

While `bdd` in Rust operates as an ultra-high-speed, zero-copy packet ingestion engine, security operations centers (SOC) and site reliability engineers (SRE) require stateful correlation and command-line resolution:

```
# Launch interactive terminal monitor with high-visibility ANSI badges:
sudo python3 contrib/python/bdd_netlink_proc.py

# Filter only process execution events (EXEC) in real time:
sudo python3 contrib/python/bdd_netlink_proc.py --event EXEC

# Stream machine-readable NDJSON for SIEM ingestion:
sudo python3 contrib/python/bdd_netlink_proc.py --json
```

7. Non-8-Bit-Aligned Bitstreams in the CLI & Engine

A natural question when dealing with bitstream data is: **Can `bdd` handle streams and units that are not aligned to 8-bit byte boundaries?**

Yes, completely. `bdd` is built from the ground up as a sub-byte bitstream processor. Alignment to 8-bit byte boundaries is never required:

1. **Arbitrary Bit Offsets:** An input stream can begin at any bit offset using `--input-skip-bits=N` (or `N :` `unit` in stream arrow syntax). For example, `11 : 13` skips 11 bits and begins streaming 13-bit units across byte boundaries.
2. **Dense Sub-Byte & Odd-Width Units:** Units of any bit width (e.g., 1-bit flags, 3-bit opcodes, 5-bit fields, 12-bit ADC readings, 13-bit PIDs) pack continuously across byte boundaries without inter-unit padding. For instance, eight 3-bit units pack tightly into exactly 3 bytes (24 bits).
3. **Periodic Bit Gaps:** You can skip arbitrary unaligned bit gaps between units (`--input-gap=5` or `+5`).
4. **End-of-Stream (EOF) Alignment Controls:**
 - **Default Zero-Padding on Output:** Unix files, fifos, and pipes operate strictly on 8-bit bytes. When writing out an unaligned bitstream (e.g., 13 bits total), `bdd` zero-pads the remaining bits of the final byte (`101...000`) so that POSIX byte writes remain valid.
 - `--drop-partial-eof` : When reading an input stream whose final unit is truncated (e.g., only 5 bits remaining when 12 bits are expected), `--drop-partial-eof` discards the incomplete trailing bits instead of zero-padding them into a spurious final unit.
 - `--input-assert-aligned` : Strictly verifies that the input stream terminates exactly on an 8-bit byte boundary, aborting with exit code 1 (`Non-aligned end of file`) if stray unaligned bits remain at EOF.

8. Multi-Stream Coordination: Merging, Interleaving & Demuxing

Round-Robin Stream Merging (`--merge-file`)

`bdd` can interleave data from one or more secondary files round-robin with the primary stream:

```
# Interleave 32-bit counter addresses with 8-bit payload bytes:
bdd --input-file=payload.bin --input-unit=8 \
    --merge-file=addresses.bin --merge-unit=32 \
    --output-hex
```

Channel Demuxing & Splitting (`--demux` , `--demux-files`)

Split multi-channel streams into separate physical files in a single pass:

```
# Slices 32-bit stereo audio into Left and Right channel files:
bdd "16S16S" --demux 0:left.raw --demux 1:right.raw < stereo.raw
```

9. Taming Bit Reversals & Endianness

Bit reversals across arbitrary bit widths frequently cause confusion. `bdd` cleanly isolates reversals into four distinct operational layers:

```
[ Input Byte Stream ]
    |
    (1)  └─> --reverse-input-bytes    (Reverses bits within each 8-bit byte read)
    ▼
[ Input Units ]
    |
    (2)  └─> --reverse-input-units    (Reverses all bits across the entire unit)
    ▼
[ Tuple Fields ]
    |
    (3)  └─> Lowercase Pattern Codes (u, s, m, c - reverses bits within specific field)
    ▼
[ Output Units ]
    |
    (4)  └─> --reverse-output-units   (Reverses all bits across the entire output unit)
    ▼
[ Output Byte Stream ]
    |
    (5)  └─> --reverse-output-bytes   (Reverses bits within each 8-bit byte written)
```

- `--input-little-endian` : Shorthand combining `--reverse-input-bytes` and `--reverse-input-units` .
- `--output-little-endian` : Shorthand combining `--reverse-output-bytes` and `--reverse-output-units` .

10. Exabyte Scale, Fast Hardware Seeking & Size Expressions

Can `bdd` skip over gigabytes of data on the command line? **Yes, without limit:**

- **Full 64-Bit Architecture (`u64`):** All skips, offsets, gaps, units, and record counts are represented internally as 64-bit unsigned integers. `bdd` can represent bit offsets up to $2^{64} - 1 \approx 1.84 \times 10^{19}$ bits, which equals **2.3 Exabytes** (2,305,843,009 Gigabytes).
- **Filesystem Seek Limits:** On seekable files, Linux `lseek64` supports offsets up to $2^{63} - 1$ bytes = **9.22 Exabytes**, executed in $O(1)$ constant time (microseconds) without memory overhead.
- **Human-Friendly Size Suffixes:** All numeric arguments accept standard scale suffixes:
 - **Binary multiples:** `K`, `M`, `G`, `T`, `P`, `E` (or `Ki`, `Mi`, `Gi`, `Ti`, `Pi`, `Ei`).
 - **Decimal multiples:** `KB`, `MB`, `GB`, `TB`, `PB`, `EB`.
 - **Byte-scaled bit skips:** On bit options (`--input-skip-bits`, `--merge-skip-bits`), specifying `B` (e.g. `10GiB`, `4GB`, `100B`) automatically multiplies bytes by 8 bits.
- **Multiplication Expressions:** All size and count options support multiplication expressions using `*` or `x`:
 - **2D/3D dimensions and frame strides:** `--input-skip-bits=1920x1080*24` or `--input-skip-bits="1920 * 1080 * 3B"`.
 - **Sizing units and offsets:** `--input-unit=3*8` (24-bit unit) or `--input-offset=2*8` (16-bit offset).

```
# Instantly seek 10 GiB into a raw disk image and inspect 4 bytes:
bdd --input-file=disk.img --input-skip-bits=10GiB --count=4 --output-hex

# Skip a 1080p 24-bit uncompressed RGB frame buffer:
bdd --input-file=video.raw --input-skip-bits=1920x1080*24 --count=100 --output-hex
```

11. Discovery, AI Ergonomics & Web GUI

`bdd` provides native primitives for autonomous AI coding agents, reverse engineers, and pipeline automation:

Built-in Format Presets (`--preset`, `--list-presets`)

Instead of manually typing complex bit patterns, 19 standard protocol presets configure patterns, unit widths, and field names in one step:

```
bdd --list-presets
```

Standard presets include: `mp3-header`, `mpeg-ts`, `wav-header`, `jpeg-sof0`, `h264-nal`, `nvfp4`, `fp6-e3m2`, `fp8-e4m3`, `fp8-e5m2`, `bf16`, `fp16`, `ipv4-header`, `udp-header`, `tcp-header`, `riscv-r-type`, `proc-pagemap`, `proc-auxv`, `pci-config`, `netlink-proc-event`.

```
# Inspect MPEG-TS headers as structured JSON objects:
bdd --input-file stream.ts --input-raw-unit 188B --preset mpeg-ts --json-object --count 3
```

Pattern Explainer & Struct Code Generation (`--explain-pattern` , `--export-c` , `--export-rust`)

Inspect bit layouts or generate production-ready packed C structs and Rust struct definitions:

```
# Generate packed C struct definition with bitfields:
bdd --preset mpeg-ts --export-c

# Generate Rust struct with #[repr(C, packed)]:
bdd --preset mpeg-ts --export-rust
```

Binary Prober (`--probe`) & Cryptographic Key Discovery (`--probe-keys`)

Analyze unknown binary blobs without prior schema knowledge:

- `--probe` : Computes Shannon entropy (0..8 bits/byte), byte class distributions, and auto-correlates periodic strides (1..512 bytes).
- `--probe-visual` : Renders 2D terminal ANSI heatmaps and sparklines ().
- `--probe-keys` : Scans the bitstream using sliding-window entropy analysis to locate candidate cryptographic keys (AES-128, AES-256, ChaCha20, Ed25519) embedded inside code or padding:

```
bdd --input-file=firmware.bin --probe-keys=256
```

Model Context Protocol (MCP) Server (`--mcp`)

`bdd` includes a native JSON-RPC 2.0 stdio MCP server for direct integration with AI coding assistants (Claude Desktop, Cursor, Antigravity):

```
bdd --mcp
```

Tools exposed: `bdd_slice` , `bdd_probe` , `bdd_probe_units` , `bdd_transcode` , `bdd_generate` , `bdd_explain_pattern` , `bdd_list_presets` .

Interactive Web Application (`--serve`)

`bdd` embeds a complete, zero-dependency browser application directly into the binary:

```
bdd --serve          # Launches web UI at http://localhost:7788
bdd --serve 8080     # Custom port (aliases: --web, --gui)
```

Features drag-and-drop file inspection, live visual bit breakdown, color-coded pattern layouts, real-time CLI command generation, and multi-sink inspection.

12. Programmatic Bitstream SDKs (Rust, C & Python)

When building device drivers, multimedia decoders, or embedded firmware, you often need to manipulate unaligned bitstreams **directly in memory** without creating files, pipes, or shell subprocesses.

Does `pip install bdd` Contain the Binary?

Yes. The Python package is fully self-contained:

- It includes pure-Python implementations of `BitStreamReader` and `BitStreamWriter` that run anywhere without a compiler.
 - It automatically locates and loads the compiled high-performance Rust library (`libbdd.so`) via `ctypes` when available, seamlessly accelerating throughput to multi-gigabit speeds.
-

Rust Library API (`bdd::bits`)

Add `bdd` to your `Cargo.toml` :

```
[dependencies]
bdd = { git = "https://github.com/e-t-u/bdd.git" }
```

```

use bdd::bits::{BitStreamReader, BitStreamWriter, read_bits_u64, write_bits_u64};

fn main() → Result<(), Box<dyn std::error::Error>> {
    // 1. Reading unaligned bits across byte boundaries:
    // Extract 13-bit MPEG PID starting at bit offset 11:
    let packet = [0x47, 0x1F, 0xFF, 0x10];
    let pid = read_bits_u64(&packet, 11, 13)?;
    assert_eq!(pid, 0x1FFF);

    // 2. Sequential in-memory bitstream reading:
    // Raw 3-byte payload holding two dense 12-bit ADC samples (0xABC, 0xDEF):
    let raw_adc = [0xAB, 0xCD, 0xEF]; // 24 bits
    let mut reader = BitStreamReader::new(&raw_adc);
    let s0 = reader.read_bits(12)?; // 0xABC (2748)
    let s1 = reader.read_bits(12)?; // 0xDEF (3567)
    assert!(reader.is_empty());

    // 3. Building an unaligned bitstream with BitStreamWriter:
    let mut writer = BitStreamWriter::new();
    writer.write_bits(0b101, 3)?; // 3-bit status flag
    writer.write_bits(s0, 12)?; // 12-bit sensor sample
    writer.write_bits(1, 1)?; // 1-bit parity flag

    let (bytes, total_bits) = writer.finish();
    assert_eq!(total_bits, 16);
    assert_eq!(bytes.len(), 2);

    // 4. In-place bitfield mutation without altering neighbor bits:
    let mut header = [0xFF, 0x00];
    write_bits_u64(&mut header, 6, 4, 3)?; // write 0b0011 at bit offset 6
    assert_eq!(header[0], 0xFC);
    assert_eq!(header[1], 0xC0);

    Ok(())
}

```

C / C++ API ([include/bdd.h](#))

Link against `libbdd.so` :

```

#include "bdd.h"
#include <stdio.h>
#include <assert.h>

int main(void) {
    uint8_t packet[4] = { 0x47, 0x1F, 0xFF, 0x10 };
    uint64_t pid = 0;

    // Read 13-bit PID at bit offset 11 across byte boundaries:
    bdd_read_bits_u64(packet, sizeof(packet), 11, 13, &pid);
    printf("Extracted 13-bit PID: 0x%lX\n", (unsigned long)pid);

    // Write a 5-bit priority tag at bit offset 27 without touching other bits:
    bdd_write_bits_u64(packet, sizeof(packet), 27, 5, 0x1F);

    // Unpack multi-field structured tuple directly from memory:
    uint64_t fields[4];
    uint8_t telemetry[2] = { 0xAB, 0xCD };
    int count = bdd_unpack_buffer("3U1x2u3M", telemetry, sizeof(telemetry), 0, fields, 4);
    printf("Unpacked %d fields: [%lu, %lu, %lu, %lu]\n",
        count, fields[0], fields[1], fields[2], fields[3]);

    return 0;
}

```

Python SDK (`pip install bdd`)

```
import bdd

# 1. Unaligned buffer slicing:
data = b"\xAC\xF0" # 10101100 11110000
val = bdd.read_bits(data, bit_offset=6, bit_count=4)
print("Unaligned slice:", val) # → 3

# 2. Sequential in-memory bitstream reading:
raw_adc = b"\xAB\xCD\xEF" # two dense 12-bit samples
reader = bdd.BitStreamReader(raw_adc)
sample0 = reader.read(12) # 0xABC (2748)
sample1 = reader.read(12) # 0xDEF (3567)
assert reader.is_empty()

# 3. Packing unaligned bitstreams with BitStreamWriter:
writer = bdd.BitStreamWriter()
writer.write(0b101, bits=3) # 3-bit status
writer.write(0xABC, bits=12) # 12-bit sensor reading
writer.write(1, bits=1) # 1-bit parity
packed_bytes, exact_bits = writer.finish()
print(f"Packed {exact_bits} bits into {len(packed_bytes)} bytes: {packed_bytes.hex()}")
# → Packed 16 bits into 2 bytes: b579

# 4. Direct in-memory pattern unpacking:
fields = bdd.unpack_buffer("3U2u3M", b"\xF5", bit_offset=0)
print("Unpacked fields:", fields) # → (7, 1, 1, 3)

# 5. Native AI float format conversions:
val = bdd.get_bdd().decode_f16(0x3C00) # FP16 → 1.0
bits = bdd.get_bdd().encode_fp4(1.0) # 1.0 → NVFP4
```

13. Architecture, Performance & Benchmarks

Measured via `make bench` (`benches/throughput.rs`) on Linux x86_64:

OPERATION	TOTAL VOLUME	THROUGHPUT (BITS/S)	THROUGHPUT (BYTES/S)	NOTES
Hardware 64-bit Bit Reversal	128 Mbits	16.00 Gbps	2,000 MB/s	Direct CPU <code>u64::reverse_bits()</code>
Bignum 1024-bit Packing/Unpacking	10.2 Mbits	5.27 Gbps	658.6 MB/s	Large-block bignum bitfield packing
Bignum 256-bit Packing/Unpacking	12.8 Mbits	1.90 Gbps	237.2 MB/s	SHA-256 size field packing/unpacking
Synthetic 8-bit Linear Stream	16.0 Mbits	234.5 Mbps	29.3 MB/s	Continuous bit generation & sink flush
1-bit Single-Bit Resolution Stream	0.5 Mbits	43.3 Mbps	5.4 MB/s	Single-bit slice accumulation & packing
Bignum 1024-bit Bit Reversal	5.1 Mbits	39.4 Mbps	4.9 MB/s	Full arbitrary-precision bit reversal
Unaligned 3-bit to 8-bit Extraction	3.0 Mbits	39.0 Mbps	4.9 MB/s	Cross-byte boundary accumulation
Tuple Pipeline (2U3U3U → Rearrange)	2.0 Mbits	29.7 Mbps	3.7 MB/s	Multi-field unpack, reorder & repack

14. Command Line Options Reference (Secondary Reference)

For AI autonomous agents, programmatic wrapper scripts, and legacy workflows, all stream arrow parameters can also be configured via explicit CLI flags:

Usage: bdd [OPTIONS] [STREAM_PATTERN] [INPUT_PATTERN] [OUTPUT_PATTERN]

Arguments:

[STREAM_PATTERN]	Stream I/O pipeline (e.g. '8→3', '4U4U → {0 1} → hex', '188B[11:13] → 13')
[INPUT_PATTERN]	Input pattern for unpacking tuples (e.g. '8U', '24S', 'sync:11u,version:2u')
[OUTPUT_PATTERN]	Output pattern for packing tuples (e.g. '8U', '8U,x,8U', '8E')

Input Unit & Container Options:

-p, --input-pattern <PATTERN>	Bit pattern to unpack input (e.g. "3U1x2u3M")
-u, --input-unit <BITS>	Input unit size in bits (default: 8, or auto-inferred from pattern)
--preset <NAME>	Use built-in protocol or float preset (e.g. mp3-header, mpeg-ts, nvfp4)
--list-presets	List all built-in format presets and exit
--input-skip-bits <BITS>	Initial bit offset before first unit [default: 0]
--input-skip-units <UNITS>	Skip initial N units from input stream [default: 0]
--input-gap <BITS>	Bit gap skipped after each unit (or between raw units) [default: 0]
--input-raw-unit <BITS>	Size of repeating raw container/frame in bits
--input-offset <BITS>	Bit offset of unit inside raw unit [default: 0]
--input-assert-aligned	Error if EOF is not byte-aligned
--drop-partial-eof	Discard incomplete trailing bits at EOF instead of zero-padding
--no-seek, --do-not-seek	Globally disable seeking on all inputs (force streaming read)
--input-no-seek	Disable seeking specifically on primary input
--input-use-seek	Explicitly enable seeking on input (default: true)

Synthetic Stream Sources & Telemetry:

-c, --input-counter	Generate sequential counter numbers (0, 1, 2...)
-0, --input-zeros	Generate endless stream of zero bits
-1, --input-ones	Generate endless stream of one bits
-r, --input-random	Generate random bits from /dev/urandom
-t, --input-tuples	Read comma-separated tuple lines from text input
--input-netlink	Stream Linux kernel process events via AF_NETLINK connector (CN_IDX_PROC)
--skip <UNITS>	Skip initial N units [default: 0]
--count <COUNT>	Process at most N units (0 or omitted = infinite) [default: 0]

Bit Reversal Options:

--reverse-input-bytes	Reverse bit order within each input byte
--reverse-input-units	Reverse bit order across entire input unit
--input-little-endian	Combined byte and unit reversal for input
--reverse-output-bytes	Reverse bit order within each output byte
--reverse-output-units	Reverse bit order across entire output unit
--output-little-endian	Combined byte and unit reversal for output

Tuple Manipulators:

<code>--rearrange <FIELDS></code>	Explicit output field order by input index (e.g. "1,0", "0 1", "{0, 1 2}")
<code>--round <F,LIMIT[,MODE]></code>	Round or clamp field F (alias: <code>--cut-maxint</code>)
<code>--remove-right <F,BITS></code>	Right-shift field F by BITS
<code>--shift-right <F,BITS></code>	Right-shift field F by BITS (synonym)
<code>--shift-left <F,BITS></code>	Left-shift field F by BITS
<code>--xor <F,PARAM></code>	Bitwise XOR field F with PARAM
<code>--and <F,PARAM></code>	Bitwise AND field F with PARAM
<code>--or <F,PARAM></code>	Bitwise OR field F with PARAM
<code>--not <FIELD></code>	Bitwise NOT (inverts bits of field F)
<code>--add <F,PARAM></code>	Add PARAM to field F
<code>--sub <F,PARAM></code>	Subtract PARAM from field F
<code>--mul <F,PARAM></code>	Multiply field F by PARAM
<code>--div <F,PARAM></code>	Integer division of field F by PARAM
<code>--mod <F,PARAM></code>	Modulo of field F by PARAM
<code>--abs <FIELD></code>	Replace signed field F with <code>abs(F)</code>
<code>--sign <FIELD></code>	Replace signed field F with sign bit (0/1)
<code>--filter <F,OP,VAL></code>	Filter tuples where predicate is false (<code>=</code> , <code>≠</code> , <code><</code> , <code>≤</code> , <code>></code> , <code>≥</code>)

Output Unit & Pattern Options:

<code>--output-pattern <PATTERN></code>	Bit pattern to pack output (supports AI floats, multipliers & counter)
<code>--output-unit <BITS></code>	Output unit size in bits (default: 8, or auto-inferred from pattern)
<code>--output-raw-unit <BITS></code>	Size of repeating raw unit / container in bits on output
<code>--output-offset <BITS></code>	Bit offset of unit inside output raw unit [default: 0]
<code>--output-gap <BITS></code>	Bit gap between output raw unit containers [default: 0]
<code>--output-skip-bits <BITS></code>	Initial zero prefix bits emitted before first unit
<code>-x, --output-hex</code>	Output units as formatted hexadecimal strings
<code>-b, --output-bits</code>	Output units as ASCII bit strings ('0' and '1')
<code>-i, --output-integers</code>	Output unsigned integer per unit (one per line)
<code>-T, --output-tuples</code>	Output fields as comma-separated tuples
<code>--output-json</code>	Output tuples as newline-delimited JSON (NDJSON)
<code>--json-object</code>	Emit JSON as keyed objects instead of positional arrays
<code>--json-fields <FIELDS></code>	Explicit comma-separated field names for JSON/CSV
<code>--output-csv</code>	Output tuples as CSV
<code>--csv-header <HEADER></code>	Optional CSV column header row
<code>--output-visual</code>	Colorized ANSI terminal dump of unaligned fields

Inspection, Code Generation, Web UI & MCP:

<code>--explain-pattern [PATTERN]</code>	Analyze bit layout, byte alignment, and field breakdown
<code>--export-c</code>	Generate packed C struct definition with bitfields
<code>--export-rust</code>	Generate Rust <code>#[repr(C, packed)]</code> struct definition
<code>--probe [FILE]</code>	Inspect binary entropy, byte classes, periodic strides, and strings
<code>--probe-units</code>	Probe unit stream characteristics and entropy AFTER input stream processing
<code>--probe-visual</code>	Render visual entropy sparkline and 2D ANSI heatmap
<code>--probe-keys [SIZE]</code>	Scan unit stream for potential maximum-entropy cryptographic keys [default: 256 bits]

<code>--probe-field <INDEX></code>	Target specific tuple field index (0-based) for unit probing
after pattern unpacking	
<code>--mcp</code>	Launch native JSON-RPC 2.0 Model Context Protocol (MCP)
server	
<code>--serve [PORT]</code>	Launch interactive Web UI browser app (aliases: <code>--web</code> , <code>--gui</code>) [default: 7788]
Demuxing & Channel Splitting:	
<code>--demux <FIELD:PATH></code>	Route individual tuple field to a dedicated output file
<code>--demux-files <PATHS></code>	Comma-separated list of output files for fields 0, 1, 2...
<code>--input-repeat <COUNT></code>	Repeat input stream N times (0 = infinite)
Merge Options:	
<code>--merge-file <PATH></code>	Interleave stream from secondary file (repeatable for N-way merge)
<code>--merge-files <PATHS></code>	Comma- or space-separated list of merge files
<code>--merge-unit <BITS></code>	Bit width of each merge unit [default: 8]
<code>--merge-skip-bits <BITS></code>	Initial bit offset in merge file [default: 0]
<code>--merge-skip-units <UNITS></code>	Skip initial N units in merge file [default: 0]
<code>--merge-gap <BITS></code>	Bit gap skipped after each merge unit [default: 0]
<code>--merge-copy-first <BITS></code>	Copy initial header bits from merge file first
<code>--merge-raw-unit <BITS></code>	Size of repeating merge container in bits
<code>--merge-offset <BITS></code>	Bit offset of unit inside merge raw unit [default: 0]
<code>--merge-drop-partial-eof</code>	Discard incomplete trailing bits at EOF in merge stream
<code>--merge-no-seek</code>	Disable seeking specifically on merge file
<code>--merge-use-seek</code>	Explicitly enable seeking on merge file (default: true)
General:	
<code>--llms</code> , <code>--ai-guide</code>	Print high-density agent cheatsheet
<code>--completions <SHELL></code>	Generate shell completion script (bash, zsh, fish, powershell, elvish)
<code>-q</code> , <code>--quiet</code>	Silence non-fatal warnings and diagnostic summaries
<code>-h</code> , <code>--help</code>	Print help
<code>-V</code> , <code>--version</code>	Print version

15. Contributed Real-World Tooling ([contrib/](#))

The [contrib/](#) directory provides production-grade reference implementations, standalone shell scripts, Python ctypes tools, and native C programs demonstrating `bdd` :

- **Multimedia & Broadcast Streams** ([contrib/shell/decode_*.sh](#) , [contrib/c/decode_media.c](#) , [contrib/python/decode_media.py](#)) :
 - MP3 unaligned 32-bit frame header extraction and $O(1)$ frame seeking.
 - MPEG-TS 188-byte container striding and 13-bit PID extraction (`188B[11:13] → 13`).
 - RIFF WAV 32-bit stereo audio demuxing and 24-bit to 16-bit PCM downsampling.
 - MP4 ISO-BMFF box traversal and H.264 NAL unit header slicing (`32U1U2U5U`).
 - JPEG SOF0 geometry and 4-bit chroma nibble extraction (`4U4U`).

- **AI Microscaling Floating-Point Codecs** ([contrib/shell/decode_ai_weights.sh](#) , [contrib/c/decode_ai_weights.c](#) , [contrib/python/decode_ai_weights.py](#)):
 - Unpacks sub-byte NVIDIA Blackwell NVFP4 (4E), OCP FP6 (6E), FP8 (8E / 8M), and BF16 into IEEE floats.
 - Slices Hugging Face `.safetensors` headers in $O(1)$ seek time without Python ML frameworks.
- **Network Packet Protocol Dissection** ([contrib/shell/decode_network.sh](#) , [contrib/c/decode_network.c](#) , [contrib/python/decode_network.py](#)):
 - Slices IPv4, UDP, and TCP headers with discrete single-bit control flags (SYN , ACK , FIN , RST).
- **System Memory Forensic Key Discovery** ([contrib/shell/stream_memory_keys.sh](#) , [contrib/python/stream_memory_keys.py](#)):
 - Streams raw memory dumps (`/dev/mem` , `/proc/kcore`) in 256-bit units to locate candidate cryptographic keys.
- **Linux Kernel Telemetry & Process Lifecycles** ([contrib/python/bdd_netlink_proc.py](#) , [contrib/python/bdd_ps.py](#) , [contrib/python/bdd_top.py](#)):
 - Full-screen curses top monitor with bit-sliced USS memory metrics from `/proc/[pid]/pagemap` .
 - Real-time kernel process lifecycle monitoring via Netlink connector sockets.

See [contrib/README.md](#) for full documentation and test data files.

16. Development, Testing, Packaging & Install

Test Suite & Verification

```
make test
# or: cargo test --all-targets --all-features
```

Run benchmarks:

```
make bench
# or: cargo bench
```

Installation from Distribution Packages

- **Fedora / RHEL / CentOS / Rocky** (`.rpm` for DNF):

```
make rpm
sudo dnf install ./dist/bdd-0.5.2-1.*.rpm
```

- **Debian / Ubuntu / Linux Mint (.deb):**

```
make deb  
sudo apt install ./dist/bdd_0.5.2_amd64.deb
```

- **Rust Cargo Crate:**

```
cargo install --git https://github.com/e-t-u/bdd.git
```

- **Python Module via Pip:**

```
make python  
pip install ./dist/bdd-0.5.2-py3-none-any.whl
```

- **C SDK Archive:**

```
make c-lib
```

- **Build All Distribution Packages:**

```
make packages
```

License

GPL-3.0-or-later. Original author: Esa Turtiainen.